CS 5600

Homework 7

Minyuan Wang

Chapter 26

Homework (Simulation) This program, x86.py, allows you to see how different thread interleavings either cause or avoid race conditions. See the README for details on how the program works, then answer the questions below.

Questions 1. Let's examine a simple program, "loop.s". First, just read and understand it. Then, run it with these arguments (./x86.py -t 1 -p loop.s -i 100 -R dx) This specifies a single thread, an interrupt every 100 instructions, and tracing of register %dx. What will %dx be during the run? Use the -c flag to check your answers; the answers, on the left, show the value of the register (or memory value) after the instruction on the right has run.

```
(base) minyuanwang@Mac CS5600HW7 % python3 x86.py -t 1 -p loop.s -i 100 -R dx
ARG seed 0
ARG numthreads 1
ARG program loop.s
ARG interrupt frequency 100
ARG interrupt randomness False
ARG argv
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0
    ?
    ?   1000 sub  $1,%dx
    ?   1001 test $0,%dx
    ?   1002 jgte .top
    ?   1003 halt
(base) minyuanwang@Mac CS5600HW7 % python3 x86.py -t 1 -p loop.s -i 100 -R dx -c

ARG seed 0
ARG numthreads 1
ARG program loop.s
ARG interrupt frequency 100
ARG interrupt randomness False
ARG argv
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0
    0
   -1   1000 sub  $1,%dx
   -1   1001 test $0,%dx
   -1   1002 jgte .top
   -1   1003 halt
```

%dx starts at 0. After sub $1,%dx, it becomes -1. The test and jgte instructions don't change it. jgte .top doesn't jump back because -1 is not ≥ 0, so the loop runs only once and the thread halts with %dx = -1. There's only one thread and the program finishes in 4 instructions, well under the 100-instruction interrupt interval, so no interrupt happens and the result is the same every time.

Question 2. Same code, different flags: (./x86.py -p loop.s -t 2 -i 100 -a dx=3,dx=3 -R dx) This specifies two threads, and initializes each %dx to 3. What values will %dx see? Run with -c to check. Does the presence of multiple threads affect your calculations? Is there a race in this code?

```
(base) minyuanwang@Mac CS5600HW7 % python3 x86.py -p loop.s -t 2 -i 100 -a dx=3,
dx=3 -R dx -c
ARG seed 0
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 100
ARG interrupt randomness False
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0                Thread 1
    3
    2   1000 sub  $1,%dx
    2   1001 test $0,%dx
    2   1002 jgte .top
    1   1000 sub  $1,%dx
    1   1001 test $0,%dx
    1   1002 jgte .top
    0   1000 sub  $1,%dx
    0   1001 test $0,%dx
    0   1002 jgte .top
   -1   1000 sub  $1,%dx
   -1   1001 test $0,%dx
   -1   1002 jgte .top
   -1   1003 halt
    3   ----- Halt;Switch -----  ----- Halt;Switch -----
    2                           1000 sub  $1,%dx
    2                           1001 test $0,%dx
    2                           1002 jgte .top
    1                           1000 sub  $1,%dx
    1                           1001 test $0,%dx
    1                           1002 jgte .top
    0                           1000 sub  $1,%dx
    0                           1001 test $0,%dx
    0                           1002 jgte .top
   -1                           1000 sub  $1,%dx
   -1                           1001 test $0,%dx
   -1                           1002 jgte .top
   -1                           1003 halt
```

Answer: in each thread, %dx goes 3 → 2 → 1 → 0 → -1. The extra thread doesn't change the math, and there's no race.

How to read it:

1. **Thread 0 starts with %dx = 3.** Each sub lowers it by 1. jgte .top jumps back while %dx >= 0, so the loop runs at values 2, 1 and 0, then exits at -1.
2. **Thread 0 finishes in 13 instructions** (4 loops × 3 instructions + halt). That's under 100, so no interrupt happens before it halts.
3. **At "Halt;Switch"**, the CPU switches to Thread 1, and %dx shows **3** again. Each thread has its own saved registers, so Thread 1 starts from its own 3, not from Thread 0's -1.
4. **Thread 1 repeats the same pattern:** 2, 1, 0, -1, then halt.

Does having multiple threads change the calculations? No. Each thread has its own copy of %dx, and with interrupts every 100 instructions, the threads never interleave. Thread 0 runs to completion, then Thread 1 does.

Is there a race? No. A race needs threads to share data, like a variable in memory. This program only uses %dx, a register each thread keeps separately, so neither thread can affect the other. Even with frequent interrupts (try -i 2), the final values stay the same.

Question 3. Run this: ./x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3 This makes the interrupt interval small/random; use different seeds (-s) to see different interleavings. Does the interrupt frequency change anything?

Making the interrupt interval small and random (-i 3 -r) changes how the two threads interleave, and different seeds produce different schedules. However, it doesn't change the result: each thread's %dx still goes 3 → 2 → 1 → 0 → -1 and both threads halt at -1. This is because %dx is a register, and the OS saves and restores each thread's registers on every context switch. The threads share no memory, so there's no race, no matter how often interrupts happen.

```
(base) minyuanwang@Mac CS5600HW7 % python3 x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3 -c -s 0
ARG seed 0
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 3
ARG interrupt randomness True
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0                Thread 1
    3
    2   1000 sub  $1,%dx
    2   1001 test $0,%dx
    2   1002 jgte .top
    3   ------ Interrupt ------  ------ Interrupt ------
    2                           1000 sub  $1,%dx
    2                           1001 test $0,%dx
    2                           1002 jgte .top
    2   ------ Interrupt ------  ------ Interrupt ------
    1   1000 sub  $1,%dx
    1   1001 test $0,%dx
    2   ------ Interrupt ------  ------ Interrupt ------
    1                           1000 sub  $1,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    1   1002 jgte .top
    0   1000 sub  $1,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    1                           1001 test $0,%dx
    1                           1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
    0   1001 test $0,%dx
    0   1002 jgte .top
   -1   1000 sub  $1,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    0                           1000 sub  $1,%dx
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1   1001 test $0,%dx
   -1   1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
    0                           1001 test $0,%dx
    0                           1002 jgte .top
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1   1003 halt
    0   ----- Halt;Switch -----  ----- Halt;Switch -----
   -1                           1000 sub  $1,%dx
   -1                           1001 test $0,%dx
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1                           1002 jgte .top
   -1                           1003 halt
```

```
(base) minyuanwang@Mac CS5600HW7 % python3 x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3 -c -s 1
ARG seed 1
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 3
ARG interrupt randomness True
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0                Thread 1
    3
    2   1000 sub  $1,%dx
    3   ------ Interrupt ------  ------ Interrupt ------
    2                           1000 sub  $1,%dx
    2                           1001 test $0,%dx
    2                           1002 jgte .top
    2   ------ Interrupt ------  ------ Interrupt ------
    2   1001 test $0,%dx
    2   1002 jgte .top
    1   1000 sub  $1,%dx
    2   ------ Interrupt ------  ------ Interrupt ------
    1                           1000 sub  $1,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    1   1001 test $0,%dx
    1   1002 jgte .top
    1   ------ Interrupt ------  ------ Interrupt ------
    1                           1001 test $0,%dx
    1                           1002 jgte .top
    1   ------ Interrupt ------  ------ Interrupt ------
    0   1000 sub  $1,%dx
    0   1001 test $0,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    0                           1000 sub  $1,%dx
    0                           1001 test $0,%dx
    0                           1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
    0   1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
   -1                           1000 sub  $1,%dx
    0   ------ Interrupt ------  ------ Interrupt ------
   -1   1000 sub  $1,%dx
   -1   1001 test $0,%dx
   -1   1002 jgte .top
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1                           1001 test $0,%dx
   -1                           1002 jgte .top
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1   1003 halt
   -1   ----- Halt;Switch -----  ----- Halt;Switch -----
   -1                           1003 halt
```

```
(base) minyuanwang@Mac CS5600HW7 % python3 x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3 -c -s 2
ARG seed 2
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 3
ARG interrupt randomness True
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0                Thread 1
    3
    2   1000 sub  $1,%dx
    2   1001 test $0,%dx
    2   1002 jgte .top
    3   ------ Interrupt ------  ------ Interrupt ------
    2                           1000 sub  $1,%dx
    2                           1001 test $0,%dx
    2                           1002 jgte .top
    2   ------ Interrupt ------  ------ Interrupt ------
    1   1000 sub  $1,%dx
    2   ------ Interrupt ------  ------ Interrupt ------
    1                           1000 sub  $1,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    1   1001 test $0,%dx
    1   1002 jgte .top
    0   1000 sub  $1,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    1                           1001 test $0,%dx
    1                           1002 jgte .top
    0                           1000 sub  $1,%dx
    0   ------ Interrupt ------  ------ Interrupt ------
    0   1001 test $0,%dx
    0   1002 jgte .top
   -1   1000 sub  $1,%dx
    0   ------ Interrupt ------  ------ Interrupt ------
    0                           1001 test $0,%dx
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1   1001 test $0,%dx
   -1   1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
    0                           1002 jgte .top
   -1                           1000 sub  $1,%dx
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1   1003 halt
   -1   ----- Halt;Switch -----  ----- Halt;Switch -----
   -1                           1001 test $0,%dx
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1                           1002 jgte .top
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1                           1003 halt
```

Question 4. (List.c) Build a version of a linked list that uses hand-over-hand locking [MS04], as cited in the chapter. You should read the paper first to understand how it works, and then implement it. Measure its performance. When does a hand-over-hand list work better than a standard list as shown in the chapter?

```
(base) minyuanwang@Mac ~ % cd desktop/CS5600HW7
(base) minyuanwang@Mac CS5600HW7 % gcc -O2 -pthread -o list list.c
(base) minyuanwang@Mac CS5600HW7 % ./list 1 4 1000 2000 0
  hoh  threads=4 size=1000 lookups/thread=2000 work=0  time=0.033 s
(base) minyuanwang@Mac CS5600HW7 % for w in 0 200; do for t in 1 2 4 8; do for m in 0 1; do ./list $m $t 1000 2000 $w; done; done; done
  big  threads=1 size=1000 lookups/thread=2000 work=0  time=0.001 s
  hoh  threads=1 size=1000 lookups/thread=2000 work=0  time=0.004 s
  big  threads=2 size=1000 lookups/thread=2000 work=0  time=0.004 s
  hoh  threads=2 size=1000 lookups/thread=2000 work=0  time=0.007 s
  big  threads=4 size=1000 lookups/thread=2000 work=0  time=0.013 s
  hoh  threads=4 size=1000 lookups/thread=2000 work=0  time=0.025 s
  big  threads=8 size=1000 lookups/thread=2000 work=0  time=0.030 s
  hoh  threads=8 size=1000 lookups/thread=2000 work=0  time=0.056 s
  big  threads=1 size=1000 lookups/thread=2000 work=200  time=0.111 s
  hoh  threads=1 size=1000 lookups/thread=2000 work=200  time=0.164 s
  big  threads=2 size=1000 lookups/thread=2000 work=200  time=0.255 s
  hoh  threads=2 size=1000 lookups/thread=2000 work=200  time=0.190 s
  big  threads=4 size=1000 lookups/thread=2000 work=200  time=0.494 s
  hoh  threads=4 size=1000 lookups/thread=2000 work=200  time=0.212 s
  big  threads=8 size=1000 lookups/thread=2000 work=200  time=0.976 s
  hoh  threads=8 size=1000 lookups/thread=2000 work=200  time=0.306 s
(base) minyuanwang@Mac CS5600HW7 %
```

My results (2-core machine)

Threads	Big lock, work=0	HoH, work=0	Big lock, work=200	HoH, work=200
1	0.002 s	0.016 s	0.084 s	0.085 s
2	0.009 s	0.030 s	0.182 s	**0.110 s**
4	0.013 s	0.109 s	0.369 s	**0.308 s**
8	0.027 s	0.254 s	0.744 s	0.836 s

Your numbers will differ, and your Mac probably has more cores, so hand-over-hand should do better there with work=200.

4. What the results mean

- **With no extra work, hand-over-hand is 5–10× slower.** It takes and releases a lock at every node, while the big lock does that once per lookup. That overhead costs much more than the parallelism saves.
- **With expensive per-node work, hand-over-hand can win.** When each node takes real time, threads benefit from moving through the list at the same time, and the locking overhead becomes small in comparison. Here it was faster at 2 and 4 threads.
- **Its advantage disappears with more threads than cores.** At 8 threads on 2 cores, threads mostly wait for each other, and hand-over-hand loses again

I implemented hand-over-hand locking by giving each node its own mutex and, during traversal, locking the next node before releasing the current one. I compared it with the chapter's single-lock list, measuring lookups on a 1,000-node list with 1–8 threads. When per-node work is cheap, hand-over-hand is much slower (5–10× in my tests), because

acquiring and releasing a lock at every node costs far more than the extra parallelism gains. Hand-over-hand only wins when visiting each node is expensive (simulated with a busy loop) and there are enough CPU cores for multiple threads to run in the list at once. In that case threads can move through the list together instead of waiting for one global lock. So hand-over-hand works better for long lists with expensive per-node operations on multi-core machines. For typical short operations, a single lock is simpler and faster.

Question 5.(bst.c) Pick your favorite data structure, such as a B-tree or other slightly more interesting structure. Implement it, and start with a simple locking strategy such as a single lock. Measure its performance as the number of concurrent threads increases.

```
(base) minyuanwang@Mac CS5600HW7 % gcc -O2 -pthread -o bst bst.c
(base) minyuanwang@Mac CS5600HW7 % for t in 1 2 4 8; do ./bst 0 $t 500000 10; done
mutex  threads=1 ops/thread=500000 inserts=10%  time=0.075 s  throughput=6.65 Mops/s
mutex  threads=2 ops/thread=500000 inserts=10%  time=0.226 s  throughput=4.43 Mops/s
mutex  threads=4 ops/thread=500000 inserts=10%  time=0.980 s  throughput=2.04 Mops/s
mutex  threads=8 ops/thread=500000 inserts=10%  time=2.099 s  throughput=1.91 Mops/s
(base) minyuanwang@Mac CS5600HW7 % for t in 1 2 4 8; do ./bst 1 $t 500000 10; done
rwlock threads=1 ops/thread=500000 inserts=10%  time=0.065 s  throughput=7.73 Mops/s
rwlock threads=2 ops/thread=500000 inserts=10%  time=0.577 s  throughput=1.73 Mops/s
rwlock threads=4 ops/thread=500000 inserts=10%  time=1.614 s  throughput=1.24 Mops/s
rwlock threads=8 ops/thread=500000 inserts=10%  time=4.848 s  throughput=0.83 Mops/s
(base) minyuanwang@Mac CS5600HW7 %
```

Threads	Single mutex (Mops/s)	Reader-writer lock (Mops/s)
1	6.65	7.73
2	4.43	1.73
4	2.04	1.24
8	1.91	0.83

What they show:

- **Single mutex:** throughput falls steadily as you add threads, from 6.65 to 1.91 Mops/s, about 3.5× worse at 8 threads. More threads means more waiting on the one lock, and no real parallel work.
- **Reader-writer lock:** it's slightly faster with 1 thread, but falls apart faster than the mutex: about 9× worse at 8 threads. The 10% of operations that are inserts block every reader. On top of that, each lookup is so short that the extra bookkeeping of the reader-writer lock costs more than letting readers share saves.

I implemented a binary search tree protected by a single mutex and measured its throughput with 1, 2, 4 and 8 threads, using 90% lookups and 10% inserts on a tree prefilled with 100,000 keys. Throughput was highest with one thread (6.65 million ops/sec) and fell steadily as threads were added: 4.43 with 2 threads, 2.04 with 4, and 1.91 with 8. Because every operation needs the same lock, threads can't work in parallel. Instead they wait for the lock and pay the cost of passing it between cores, so adding threads makes performance worse. I also tried a single reader-writer lock, which was slightly faster with one thread (7.73) but degraded even more (0.83 with 8 threads). Each lookup is very short, so the reader-writer lock's extra overhead outweighs the benefit of sharing, and the 10% inserts still block all readers. To scale, the tree would need finer-grained locking (for example, per-node locks) or a lock-free design.

Question 6. Finally, think of a more interesting locking strategy for this favorite data structure of yours. Implement it, and measure its performance. How does it compare to the straightforward locking approach?

I implemented two finer-grained locking strategies for my binary search tree and compared them with the single-mutex version from Question 5, using 90% lookups and 10% inserts on a tree prefilled with 100,000 keys.

1. **Hand-over-hand locking:** each node has its own lock. While walking down the tree, a thread locks the child before releasing the parent.
2. **Partitioned locking:** the key range is split into 16 contiguous ranges, each stored in its own subtree with its own mutex. An operation locks only the subtree its key belongs to, and the overall structure stays ordered.

Throughput (millions of ops/sec) with 1, 2, 4 and 8 threads:

Threads	Single mutex	Hand-over-hand	Partitioned
1	5.87	5.26	8.34
2	3.50	3.81	8.00
4	1.21	0.76	7.63
8	1.00	0.37	6.69

The single mutex doesn't scale: throughput drops by about 6× from 1 to 8 threads, because every operation competes for the same lock. Hand-over-hand locking performed even

worse (0.37 Mops/s at 8 threads). Every operation must first lock the root, so the root lock becomes a bottleneck just like the single mutex. In addition, each operation acquires and releases about 20 locks on its way down instead of one. The partitioned tree performed far better. Throughput stayed nearly flat (8.34 → 6.69 Mops/s), and it was 6.7× faster than the single mutex at 8 threads. With 16 independent locks, threads rarely contend, and each subtree is smaller, so even single-threaded operations were faster. Scaling still isn't perfect, because threads sometimes collide on the same subtree and not all CPU cores are equally fast. Overall, splitting the data so that threads usually need different locks helped much more than adding more locks along the same path.

Chapter 28

Homework (Simulation) This program, x86.py, allows you to see how different thread interleavings either cause or avoid race conditions. See the README for details on how the program works and answer the questions below.

Questions 1. Examine flag.s. This code "implements" locking with a single memory flag. Can you understand the assembly?

Yes. flag.s tries to build a lock from one shared memory word, flag, where 0 means the lock is free and 1 means it's held. The lock protects a shared counter, count. Each thread runs a loop that acquires the lock, increments count, releases the lock, and repeats until its loop counter in %bx reaches 0.

Memory layout. The two .var lines put flag at address 100 and count at address 104. The code loads at address 1000 by default.

Line by line:

asm

.acquire

mov flag, %ax   # 1000: load the lock value into ax

test $0, %ax   # 1001: compare ax with 0

jne .acquire   # 1002: if ax != 0, the lock is held, so spin back to 1000

mov $1, flag   # 1003: lock looked free, so set flag = 1 to claim it

```
# critical section
mov  count, %ax    # 1004: load count into ax
add  $1, %ax       # 1005: ax = ax + 1
mov  %ax, count    # 1006: store ax back into count

# release lock
mov  $0, flag      # 1007: set flag = 0 so other threads can enter

# loop control
sub  $1, %bx       # 1008: bx = bx - 1
test $0, %bx       # 1009: compare bx with 0
jgt  .top          # 1010: if bx > 0, run the loop again

halt               # 1011: thread finishes
```

How it's supposed to work:

1. **Acquire (1000–1003):** The thread reads flag. If it's 1, the thread spin-waits, jumping back and checking again until the lock is released. If it's 0, the thread writes 1 to take the lock.
2. **Critical section (1004–1006):** count = count + 1 takes three separate instructions (load, add, store). If two threads ran these at the same time they could lose updates, which is why the lock is needed.
3. **Release (1007):** Writing 0 to flag frees the lock.
4. **Loop (1008–1010):** %bx holds how many times this thread should increment count. Each thread has its own registers, so it's set per thread with -a, for example -a bx=5,bx=5.

Why "implements" is in quotes: the lock doesn't actually work. Testing the flag (1000–1002) and setting it (1003) are separate, non-atomic instructions, so an interrupt can arrive between them. For example:

Step	Thread 0	Thread 1	flag
1	1000: reads flag → ax = 0		0
2	1001–1002: sees 0, falls through		0
3	*interrupt*		0
4		1000–1002: reads flag = 0, falls through	0
5		1003: sets flag = 1, enters critical section	1
6		*interrupt*	1
7	1003: sets flag = 1, enters critical section		1

Both threads are now in the critical section together, so mutual exclusion is broken. If they interleave inside 1004–1006, one increment can overwrite the other and count ends up lower than the expected total (number of threads × bx).

Conclusion: flag.s shows the basic shape of a spin lock: spin until free, set, do the critical section, clear. But it fails because the test and the set aren't done as one atomic operation. A correct lock needs a hardware atomic instruction such as test-and-set or exchange, which x86.py provides as xchg. That's what the later file test-and-set.s uses.

2. When you run with the defaults, does flag.s work? Use the -M and -R flags to trace variables and registers (and turn on -c to see their values). Can you predict what value will end up in flag?

```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p flag.s -M flag,count -R ax,bx -c
ARG seed 0
ARG numthreads 2
ARG program flag.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG argv
ARG load address 1000
ARG memsize 128
ARG memtrace flag,count
ARG regtrace ax,bx
ARG cctrace False
ARG printstats False
ARG verbose False

 flag count      ax    bx          Thread 0                Thread 1
    0     0       0     0
    0     0       0     0   1000 mov  flag, %ax
    0     0       0     0   1001 test $0, %ax
    0     0       0     0   1002 jne  .acquire
    1     0       0     0   1003 mov  $1, flag
    1     0       0     0   1004 mov  count, %ax
    1     0       1     0   1005 add  $1, %ax
    1     1       1     0   1006 mov  %ax, count
    0     1       1     0   1007 mov  $0, flag
    0     1       1    -1   1008 sub  $1, %bx
    0     1       1    -1   1009 test $0, %bx
    0     1       1    -1   1010 jgt .top
    0     1       1    -1   1011 halt
    0     1       0     0   ----- Halt;Switch -----  ----- Halt;Switch -----
    0     1       0     0                           1000 mov  flag, %ax
    0     1       0     0                           1001 test $0, %ax
    0     1       0     0                           1002 jne  .acquire
```

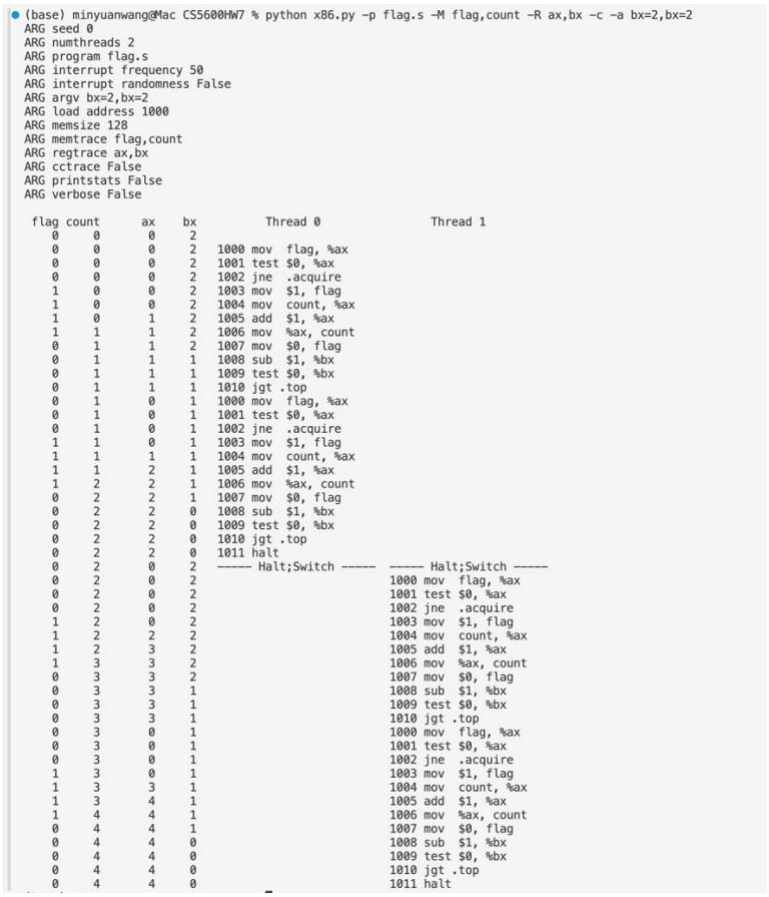
Yes, it works with the defaults: count ends at 2, which is correct.

flag ends at 0. It goes to 1 while a thread is in the critical section (1003–1006) and back to 0 at 1007.

Why it works: the threads never interleave. Each one finishes in 12 instructions, well before the 50-instruction interrupt, so they run one after the other and never race. The lock wasn't really tested here. It only looks correct because nothing interrupted the threads between checking flag and setting it.

Each thread has its own registers, so when Thread 1 starts, ax and bx reset to 0. That's why the ax and bx columns jump back to 0 at the switch.

3. Change the value of the register %bx with the -a flag (e.g., -a bx=2,bx=2 if you are running just two threads). What does the code do? How does it change your answer for the question above?



```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p flag.s -M flag,count -R ax,bx -c -a bx=2,bx=2
ARG seed 0
ARG numthreads 2
ARG program flag.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG argv bx=2,bx=2
ARG load address 1000
ARG memsize 128
ARG memtrace flag,count
ARG regtrace ax,bx
ARG cctrace False
ARG printstats False
ARG verbose False

 flag count      ax    bx          Thread 0                Thread 1
    0     0       0     2
    0     0       0     2  1000 mov  flag, %ax
    0     0       0     2  1001 test $0, %ax
    0     0       0     2  1002 jne  .acquire
    1     0       0     2  1003 mov  $1, flag
    1     0       0     2  1004 mov  count, %ax
    1     0       1     2  1005 add  $1, %ax
    1     1       1     2  1006 mov  %ax, count
    0     1       1     2  1007 mov  $0, flag
    0     1       1     1  1008 sub  $1, %bx
    0     1       1     1  1009 test $0, %bx
    0     1       1     1  1010 jgt .top
    0     1       0     1  1000 mov  flag, %ax
    0     1       0     1  1001 test $0, %ax
    0     1       0     1  1002 jne  .acquire
    1     1       0     1  1003 mov  $1, flag
    1     1       1     1  1004 mov  count, %ax
    1     1       2     1  1005 add  $1, %ax
    1     2       2     1  1006 mov  %ax, count
    0     2       2     1  1007 mov  $0, flag
    0     2       2     0  1008 sub  $1, %bx
    0     2       2     0  1009 test $0, %bx
    0     2       2     0  1010 jgt .top
    0     2       2     0  1011 halt
    0     2       0     2  ----- Halt;Switch -----  ----- Halt;Switch -----
    0     2       0     2                          1000 mov  flag, %ax
    0     2       0     2                          1001 test $0, %ax
    0     2       0     2                          1002 jne  .acquire
    1     2       0     2                          1003 mov  $1, flag
    1     2       2     2                          1004 mov  count, %ax
    1     2       3     2                          1005 add  $1, %ax
    1     3       3     2                          1006 mov  %ax, count
    0     3       3     2                          1007 mov  $0, flag
    0     3       3     1                          1008 sub  $1, %bx
    0     3       3     1                          1009 test $0, %bx
    0     3       3     1                          1010 jgt .top
    0     3       0     1                          1000 mov  flag, %ax
    0     3       0     1                          1001 test $0, %ax
    0     3       0     1                          1002 jne  .acquire
    1     3       0     1                          1003 mov  $1, flag
    1     3       3     1                          1004 mov  count, %ax
    1     3       4     1                          1005 add  $1, %ax
    1     4       4     1                          1006 mov  %ax, count
    0     4       4     1                          1007 mov  $0, flag
    0     4       4     0                          1008 sub  $1, %bx
    0     4       4     0                          1009 test $0, %bx
    0     4       4     0                          1010 jgt .top
    0     4       4     0                          1011 halt
```

Command:

python x86.py -p flag.s -M flag,count -R ax,bx -c -a bx=2,bx=2

What the code does:

- %bx is each thread's **loop counter**. Setting bx=2 makes each thread go through the acquire → increment → release loop **twice**.
- Each pass ends with sub $1, %bx, which takes bx from 2 to 1, then 1 to 0. While bx > 0, jgt .top jumps back to 1000. When bx hits 0, the jump isn't taken and the thread runs halt.
- So each thread adds 2 to count, and with 2 threads the expected total is **4**.

How this changes the Q2 answer:

- The final count goes from 2 to 4, because it's now 2 threads × 2 loops instead of 2 threads × 1.
- flag still ends at **0**, since every pass releases the lock at 1007 before looping or halting.

- The code still works, for the same reason as in Q2: each thread runs 23 instructions (11 per pass × 2, plus halt), which is still under the 50-instruction interrupt interval. Thread 0 finishes before Thread 1 starts, so the threads never interleave. The race between checking flag (1000) and setting it (1003) still never gets a chance to happen.

4. Set bx to a high value for each thread, and then use the -i flag to generate different interrupt frequencies; what values lead to a bad outcomes? Which lead to good outcomes?

- When it goes wrong: the only dangerous moment is an interrupt after a thread reads flag (1000) but before it sets it (1003). The other thread then also sees the lock as free, both enter the critical section, and increments get lost. An interrupt *inside* the critical section on its own is harmless, because the other thread sees flag = 1 and just spins.
- Small intervals (1–10) are almost always bad. Interrupts come so often that they keep landing in that window.
- Multiples of 11 are always good (11, 22, 33, 44, …). One loop pass is exactly 11 instructions (1000–1010). With an interval that's a multiple of 11, each thread is always interrupted at the end of a pass, after it has released the lock, so it never stops in the middle of acquiring it.
- Large intervals (≥ 110) are good. Each thread needs 11 × 10 + 1 = 111 instructions to finish. Thread 0 runs basically to completion before Thread 1 does anything that matters, so the threads effectively don't interleave.
- Some other values (15, 16, 21, 23, 30) happen to be good. With these, the interrupts just never landed between 1000 and 1003. That's luck from where the interrupts fell, not a sign the lock works.

Conclusion: whether flag.s works depends entirely on when the interrupts happen. It's correct only when no interrupt falls between the check (1000) and the set (1003). Since a real OS can interrupt anywhere, this lock is unsafe. It needs an atomic test-and-set (xchg) so the check and the set happen as one step.

5. Now let's look at the program test-and-set.s. First, try to understand the code, which uses the xchg instruction to build a simple locking primitive. How is the lock acquire written? How about lock release?

Code and addresses (mutex is at address 100, count at 104, and the code loads at 1000):

```
asm
.acquire
mov  $1, %ax      # 1000: ax = 1 (the value we want to put in the lock)
xchg %ax, mutex    # 1001: atomically swap ax and mutex
test $0, %ax      # 1002: compare the OLD mutex value with 0
jne  .acquire     # 1003: old value was 1 → lock was held, spin and retry

# critical section
mov  count, %ax    # 1004
add  $1, %ax      # 1005
mov  %ax, count    # 1006

# release lock
mov  $0, mutex     # 1007: mutex = 0

sub  $1, %bx      # 1008: loop control
test $0, %bx      # 1009
jgt .top          # 1010
halt              # 1011
```

Lock acquire (1000–1003):

1. mov $1, %ax puts 1 in ax.
2. xchg %ax, mutex swaps ax and mutex in **one instruction**: the old mutex value goes into ax, and 1 goes into mutex. In x86.py this is atomic_exchange(), which does the read and the write as a single step.
3. test $0, %ax / jne .acquire check the **old** value:
    - **Old value 0:** the lock was free, and this thread just set it to 1, so it now owns the lock and enters the critical section.

- **Old value 1:** someone else holds the lock. Writing 1 over 1 changes nothing, so the thread jumps back and tries again (spin-waiting).

This is a **test-and-set spin lock**. The "test" (getting the old value) and the "set" (writing 1) happen together.

Lock release (1007):

- mov $0, mutex writes 0 to the lock. It doesn't need to be atomic: only the thread holding the lock releases it, and a single store can't be interrupted partway.

Trace showing it works (-a bx=1,bx=1 -i 2 -M mutex,count -R ax,bx -c):

mutex count ax bx Thread 0 Thread 1

0 0 1 1 1000 mov $1, %ax

1 0 0 1 1001 xchg %ax, mutex ← T0 gets old value 0 → owns lock

------ Interrupt ------

1 0 1 1 1000 mov $1, %ax

1 0 1 1 1001 xchg %ax, mutex ← T1 gets old value 1

------ Interrupt ------

1 0 0 1 1002 test $0, %ax

1 0 0 1 1003 jne .acquire ← ax=0, falls through into CS

------ Interrupt ------

1 0 1 1 1002 test $0, %ax

1 0 1 1 1003 jne .acquire ← ax=1, spins

------ Interrupt ------

1 0 0 1 1004 mov count, %ax ← only T0 is in the critical section

Even with an interrupt every 2 instructions, landing right after Thread 0's xchg, Thread 1 gets 1 back from its own xchg and spins. Only one thread can ever get 0 back.

6. Now run the code, changing the value of the interrupt interval (-i) again, and making sure to loop for a number of times. Does the code always work as expected? Does it sometimes lead to an inefficient use of the CPU? How could you quantify that?

Setup: bx=10 for both threads, so the correct count is 20.

python x86.py -p test-and-set.s -M count -c -a bx=10,bx=10 -i 5

1. Does it always work?
Yes. Every interval I tried (-i 1 to 200) ended with count = 20. Because xchg checks and sets the lock in one atomic instruction, only one thread can be in the critical section at a time, wherever the interrupts land. The results are below.

2. Is the CPU used inefficiently?
Yes, often. Suppose a thread is interrupted while it holds the lock. The other thread then spends its whole time slice spin-waiting: it runs mov → xchg → test → jne over and over, gets 1 back every time, and makes no progress. It burns CPU until the next interrupt gives the lock holder a chance to finish and release.

3. How to quantify it:
Count the instructions spent spinning. Without any contention, each thread runs 10 loop passes × 11 instructions + 1 halt = 111 instructions, so the two threads need 222 instructions in total. Anything beyond that is wasted spinning. Equivalently, count the failed acquire attempts (xchg calls that returned 1). Each one costs 4 instructions (1000–1003).

7. Use the -P flag to generate specific tests of the locking code. For example, run a schedule that grabs the lock in the first thread, but then tries to acquire it in the second. Does the right thing happen? What else should you test?

```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p test-and-set.s -M mutex,count -R ax,bx -c -a bx=1,bx=1 -i 5
ARG seed 0
ARG numthreads 2
ARG program test-and-set.s
ARG interrupt frequency 5
ARG interrupt randomness False
ARG argv bx=1,bx=1
ARG load address 1000
ARG memsize 128
ARG memtrace mutex,count
ARG regtrace ax,bx
ARG cctrace False
ARG printstats False
ARG verbose False

mutex count      ax    bx          Thread 0                Thread 1
    0     0       0     1
    0     0       1     1   1000 mov  $1, %ax
    1     0       0     1   1001 xchg %ax, mutex
    1     0       0     1   1002 test $0, %ax
    1     0       0     1   1003 jne  .acquire
    1     0       0     1   1004 mov  count, %ax
    1     0       0     1   ------ Interrupt ------  ------ Interrupt ------
    1     0       1     1                           1000 mov  $1, %ax
    1     0       1     1                           1001 xchg %ax, mutex
    1     0       1     1                           1002 test $0, %ax
    1     0       1     1                           1003 jne  .acquire
    1     0       1     1                           1000 mov  $1, %ax
    1     0       0     1   ------ Interrupt ------  ------ Interrupt ------
    1     0       1     1   1005 add  $1, %ax
    1     1       1     1   1006 mov  %ax, count
    0     1       1     1   1007 mov  $0, mutex
    0     1       1     0   1008 sub  $1, %bx
    0     1       1     0   1009 test $0, %bx
    0     1       1     1   ------ Interrupt ------  ------ Interrupt ------
    1     1       0     1                           1001 xchg %ax, mutex
    1     1       0     1                           1002 test $0, %ax
    1     1       0     1                           1003 jne  .acquire
    1     1       1     1                           1004 mov  count, %ax
    1     1       2     1                           1005 add  $1, %ax
    1     1       1     0   ------ Interrupt ------  ------ Interrupt ------
    1     1       1     0   1010 jgt .top
    1     1       1     0   1011 halt
    1     1       2     1   ----- Halt;Switch -----  ----- Halt;Switch -----
    1     2       2     1                           1006 mov  %ax, count
    0     2       2     1                           1007 mov  $0, mutex
    0     2       2     0                           1008 sub  $1, %bx
    0     2       2     0   ------ Interrupt ------  ------ Interrupt ------
    0     2       2     0                           1009 test $0, %bx
    0     2       2     0                           1010 jgt .top
    0     2       2     0                           1011 halt
```

Test: T0 grabs the lock, then T1 tries to acquire it (-i 5, bx=1,bx=1):

mutex count Thread 0 Thread 1

1 0 1001 xchg → gets 0, owns lock

1 0 1004 (in critical section)

------ Interrupt ------

1 0 1001 xchg → gets 1, spins

------ Interrupt ------

0 1 1007 release

------ Interrupt ------

1 1 1001 xchg → gets 0, owns lock

0 2 1007 release

Yes, the right thing happens. T1 spins while T0 holds the lock, and it only enters after T0 releases it. The final count is 2, which is correct.

What else to test:

1. Both threads acquire at the same moment (-i 1, switching every instruction). Only one xchg returns 0, so only one thread gets in. ✓
2. Interrupt right after xchg, before test (-i 2). The lock is already held at that point, so the other thread spins. ✓
3. Interrupt inside the critical section. The other thread spins and doesn't touch count. ✓
4. More threads (-t 3 -a bx=10). count = 30 for every -i I tried. ✓
5. Efficiency. Measure the instructions wasted on spinning (20–39% in Q6).
6. Fairness. Test-and-set doesn't guarantee a waiting thread will ever get the lock, so it can starve. A ticket lock fixes this.

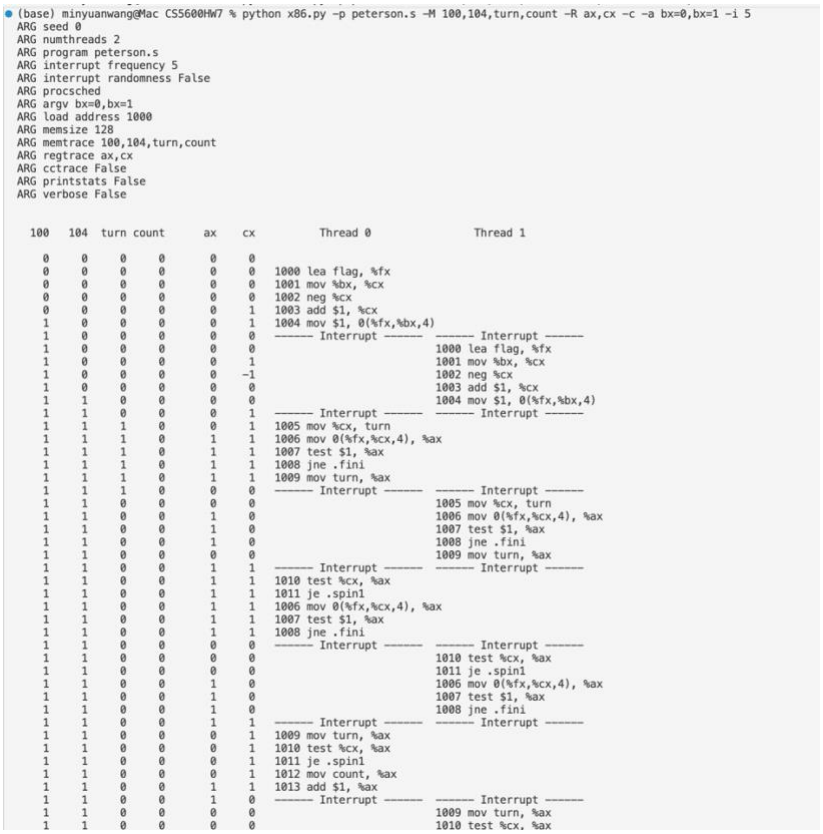
8. Now let's look at the code in peterson.s, which implements Peterson's algorithm (mentioned in a sidebar in the text). Study the code and see if you can make sense of it.

Idea: Peterson's algorithm makes a lock for 2 threads using only ordinary loads and stores, with no atomic instruction like xchg. It uses two shared variables:

flag[0], flag[1]: "thread i wants to enter."

turn: whose turn it is to go first when both want in.

9. Now run the code with different values of -i. What kinds of different behavior do you see? Make sure to set the thread IDs appropriately (using -a bx=0,bx=1 for example) as the code assumes it.



```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -i 5
ARG seed 0
ARG numthreads 2
ARG program peterson.s
ARG interrupt frequency 5
ARG interrupt randomness False
ARG procsched
ARG argv bx=0,bx=1
ARG load address 1000
ARG memsize 128
ARG memtrace 100,104,turn,count
ARG regtrace ax,cx
ARG cctrace False
ARG printstats False
ARG verbose False

  100   104  turn count      ax    cx          Thread 0                Thread 1

    0     0     0     0       0     0
    0     0     0     0       0     0   1000 lea flag, %fx
    0     0     0     0       0     0   1001 mov %bx, %cx
    0     0     0     0       0     0   1002 neg %cx
    0     0     0     0       0     1   1003 add $1, %cx
    1     0     0     0       0     1   1004 mov $1, 0(%fx,%bx,4)
    1     0     0     0       0     0   ------ Interrupt ------  ------ Interrupt ------
    1     0     0     0       0     0                           1000 lea flag, %fx
    1     0     0     0       0     1                           1001 mov %bx, %cx
    1     0     0     0       0    -1                           1002 neg %cx
    1     0     0     0       0     0                           1003 add $1, %cx
    1     1     0     0       0     0                           1004 mov $1, 0(%fx,%bx,4)
    1     1     0     0       0     1   ------ Interrupt ------  ------ Interrupt ------
    1     1     1     0       0     1   1005 mov %cx, turn
    1     1     1     0       1     1   1006 mov 0(%fx,%cx,4), %ax
    1     1     1     0       1     1   1007 test $1, %ax
    1     1     1     0       1     1   1008 jne .fini
    1     1     1     0       1     1   1009 mov turn, %ax
    1     1     1     0       0     0   ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1005 mov %cx, turn
    1     1     0     0       1     0                           1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     0                           1007 test $1, %ax
    1     1     0     0       1     0                           1008 jne .fini
    1     1     0     0       0     0                           1009 mov turn, %ax
    1     1     0     0       1     1   ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     1   1010 test %cx, %ax
    1     1     0     0       1     1   1011 je .spin1
    1     1     0     0       1     1   1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     1   1007 test $1, %ax
    1     1     0     0       1     1   1008 jne .fini
    1     1     0     0       0     0   ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1010 test %cx, %ax
    1     1     0     0       0     0                           1011 je .spin1
    1     1     0     0       1     0                           1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     0                           1007 test $1, %ax
    1     1     0     0       1     0                           1008 jne .fini
    1     1     0     0       1     1   ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     1   1009 mov turn, %ax
    1     1     0     0       0     1   1010 test %cx, %ax
    1     1     0     0       0     1   1011 je .spin1
    1     1     0     0       0     1   1012 mov count, %ax
    1     1     0     0       1     1   1013 add $1, %ax
    1     1     0     0       1     0   ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1009 mov turn, %ax
    1     1     0     0       0     0                           1010 test %cx, %ax
```

Command:

python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -i 5

(flag[0] is at address 100 and flag[1] is at 104.)

Results. With no waiting, each thread needs 15 instructions, so 30 in total.

-i	count	total instrs	spinning
1, 2	2	45	15
5	2	54	24
7–9	2	33	3
≥ 15	2	30	0

Behaviors:

1. Always correct. count = 2 for every -i, so mutual exclusion holds without any atomic instruction.

2. No contention (-i ≥ 15). Thread 0 finishes before Thread 1 starts. Thread 1 sees flag[0] = 0 and enters right away.
3. Contention (small -i). Both threads set their flags to 1. Whichever one writes turn last gives way and spins, while the other enters. For example, at -i 5, Thread 1 sets turn = 0 last, so Thread 0 enters first and Thread 1 waits until flag[0] = 0.
4. Wasted CPU. The waiting thread spins in .spin1/.spin2, which costs between 3 and 24 extra instructions depending on where the interrupts land.

Conclusion: Peterson's algorithm is always correct for 2 threads. The interrupt interval only changes how much time is wasted spin-waiting.

10. Can you control the scheduling (with the -P flag) to "prove" that the code works? What are the different cases you should show hold? Think about mutual exclusion and deadlock avoidance.

Case 1: No contention
`-P 00000000000000011111111111111111` (Thread 0 runs all 15 instructions, then Thread 1)

- Thread 0 sees `flag[1]` = 0 and enters immediately. Later, Thread 1 sees `flag[0]` = 0 and also enters immediately.
- ✓ A thread that's alone gets the lock without waiting.

Case 2: One thread in the critical section, the other tries to enter (mutual exclusion)
`-P 00000000001111111111111000000000111111`

- Thread 0 is in the critical section (1012) when Thread 1 sets `flag[1]=1` and `turn=0`.
- Thread 1 sees `flag[0]=1` and `turn=0`, so it **spins** (1011 `je .spin1`).
- Thread 1 enters only after Thread 0 releases (1015, `flag[0]=0`). Final `count` = 2.
- ✓ Mutual exclusion holds, and the waiting thread gets in once the lock is free.

Case 3: Both threads try at exactly the same time (`-P 01`, switching every instruction)

- Both flags are 1, both threads write `turn`, and both reach the spin loop.
- Thread 1 wrote `turn` last (`turn=0`), so **Thread 0 enters** and Thread 1 spins until Thread 0 releases.
- ✓ No deadlock: `turn` can hold only one value, so exactly one thread can exit the spin loop.

Case 4: The order of the `turn` writes decides the winner

Schedule	Who writes `turn` last	Who enters first
`-P 00000111111000000000000000001111111111111111111`	Thread 0 (`turn=1`)	**Thread 1**
`-P 00000011111100000000000000001111111111111111111`	Thread 1 (`turn=0`)	**Thread 0**

- ✓ The thread that writes `turn` last gives way. Either thread can win, but **never both**, and **never neither**.

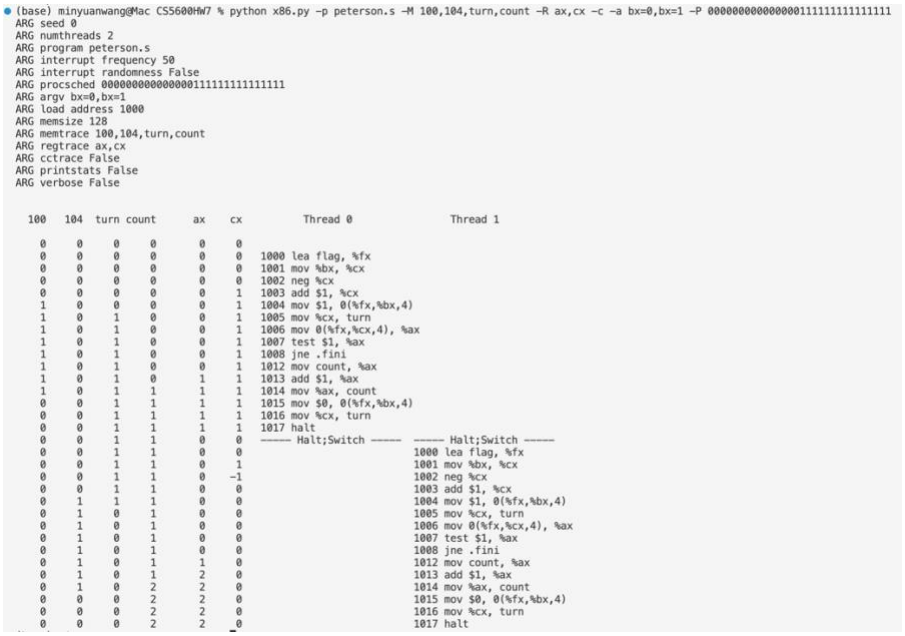
Results: every schedule ended with `count` = 2 and both flags 0, so the lock was correctly released.

Properties shown:

1. **Mutual exclusion:** two threads are never in the critical section at once (Cases 2–4).
2. **No deadlock / progress:** when both want the lock, one always gets in, because `turn` breaks the tie (Cases 3 and 4).
3. **No unnecessary waiting:** a thread that's alone never spins (Case 1).
4. **Bounded waiting:** a waiting thread enters as soon as the other releases (Case 2). It can't be passed over forever, because a thread trying again would set `turn` to the other thread and give it priority.

Case 1: No contention

python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 00000000000000001111111111111111



```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 00000000000000001111111111111111
ARG seed 0
ARG numthreads 2
ARG program peterson.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG procsched 00000000000000001111111111111111
ARG argv bx=0,bx=1
ARG load address 1000
ARG memsize 128
ARG memtrace 100,104,turn,count
ARG regtrace ax,cx
ARG cctrace False
ARG printstats False
ARG verbose False

  100   104  turn count     ax    cx         Thread 0              Thread 1

    0     0     0     0      0     0
    0     0     0     0      0     0  1000 lea flag, %fx
    0     0     0     0      0     0  1001 mov %bx, %cx
    0     0     0     0      0     0  1002 neg %cx
    0     0     0     0      0     1  1003 add $1, %cx
    1     0     0     0      0     1  1004 mov $1, 0(%fx,%bx,4)
    1     0     1     0      0     1  1005 mov %cx, turn
    1     0     1     0      0     1  1006 mov 0(%fx,%cx,4), %ax
    1     0     1     0      0     1  1007 test $1, %ax
    1     0     1     0      0     1  1008 jne .fini
    1     0     1     0      0     1  1012 mov count, %ax
    1     0     1     0      1     1  1013 add $1, %ax
    1     0     1     1      1     1  1014 mov %ax, count
    0     0     1     1      1     1  1015 mov $0, 0(%fx,%bx,4)
    0     0     1     1      1     1  1016 mov %cx, turn
    0     0     1     1      1     1  1017 halt
    0     0     1     1      0     0  ----- Halt;Switch -----  ----- Halt;Switch -----
    0     0     1     1      0     0                           1000 lea flag, %fx
    0     0     1     1      0     1                           1001 mov %bx, %cx
    0     0     1     1      0    -1                           1002 neg %cx
    0     0     1     1      0     0                           1003 add $1, %cx
    0     1     1     1      0     0                           1004 mov $1, 0(%fx,%bx,4)
    0     1     0     1      0     0                           1005 mov %cx, turn
    0     1     0     1      0     0                           1006 mov 0(%fx,%cx,4), %ax
    0     1     0     1      0     0                           1007 test $1, %ax
    0     1     0     1      0     0                           1008 jne .fini
    0     1     0     1      1     0                           1012 mov count, %ax
    0     1     0     1      2     0                           1013 add $1, %ax
    0     1     0     2      2     0                           1014 mov %ax, count
    0     0     0     2      2     0                           1015 mov $0, 0(%fx,%bx,4)
    0     0     0     2      2     0                           1016 mov %cx, turn
    0     0     0     2      2    -0                           1017 halt
```

Case 2: Thread 0 is in the critical section, Thread 1 tries

python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 00000000001111111111111100000000000111111

```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 00000000001111111111111100000000000111111
ARG seed 0
ARG numthreads 2
ARG program peterson.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG procsched 00000000001111111111111100000000000111111
ARG argv bx=0,bx=1
ARG load address 1000
ARG memsize 128
ARG memtrace 100,104,turn,count
ARG regtrace ax,cx
ARG cctrace False
ARG printstats False
ARG verbose False


  100   104  turn count      ax    cx          Thread 0                Thread 1

    0     0     0     0       0     0
    0     0     0     0       0     0  1000 lea flag, %fx
    0     0     0     0       0     0  1001 mov %bx, %cx
    0     0     0     0       0     0  1002 neg %cx
    0     0     0     0       0     1  1003 add $1, %cx
    1     0     0     0       0     1  1004 mov $1, 0(%fx,%bx,4)
    1     0     1     0       0     1  1005 mov %cx, turn
    1     0     1     0       0     1  1006 mov 0(%fx,%cx,4), %ax
    1     0     1     0       0     1  1007 test $1, %ax
    1     0     1     0       0     1  1008 jne .fini
    1     0     1     0       0     1  1012 mov count, %ax
    1     0     1     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    1     0     1     0       0     0                           1000 lea flag, %fx
    1     0     1     0       0     1                           1001 mov %bx, %cx
    1     0     1     0       0    -1                           1002 neg %cx
    1     0     1     0       0     0                           1003 add $1, %cx
    1     1     1     0       0     0                           1004 mov $1, 0(%fx,%bx,4)
    1     1     0     0       0     0                           1005 mov %cx, turn
    1     1     0     0       1     0                           1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     0                           1007 test $1, %ax
    1     1     0     0       1     0                           1008 jne .fini
    1     1     0     0       0     0                           1009 mov turn, %ax
    1     1     0     0       0     0                           1010 test %cx, %ax
    1     1     0     0       0     0                           1011 je .spin1
    1     1     0     0       1     0                           1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     0                           1007 test $1, %ax
    1     1     0     0       0     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     1  1013 add $1, %ax
    1     1     0     1       1     1  1014 mov %ax, count
    0     1     0     1       1     1  1015 mov $0, 0(%fx,%bx,4)
    0     1     1     1       1     1  1016 mov %cx, turn
    0     1     1     1       1     1  1017 halt
    0     1     1     1       1     0  ----- Halt;Switch -----  ----- Halt;Switch -----
    0     1     1     1       1     0                           1008 jne .fini
    0     1     1     1       1     0                           1009 mov turn, %ax
    0     1     1     1       1     0                           1010 test %cx, %ax
    0     1     1     1       1     0                           1011 je .spin1
    0     1     1     1       1     0                           1012 mov count, %ax
    0     1     1     1       2     0                           1013 add $1, %ax
    0     1     1     2       2     0                           1014 mov %ax, count
    0     0     1     2       2     0                           1015 mov $0, 0(%fx,%bx,4)
    0     0     0     2       2     0                           1016 mov %cx, turn
    0     0     0     2       2     0                           1017 halt
```

Case 3: Both try at the same time

python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 01

```
(base) minyuanwang@Mac CS5600HW7 % python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 01
ARG seed 0
ARG numthreads 2
ARG program peterson.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG procsched 01
ARG argv bx=0,bx=1
ARG load address 1000
ARG memsize 128
ARG memtrace 100,104,turn,count
ARG regtrace ax,cx
ARG cctrace False
ARG printstats False
ARG verbose False


  100   104  turn count      ax    cx          Thread 0                Thread 1

    0     0     0     0       0     0
    0     0     0     0       0     0  1000 lea flag, %fx
    0     0     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0     0                           1000 lea flag, %fx
    0     0     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0     0  1001 mov %bx, %cx
    0     0     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0     1                           1001 mov %bx, %cx
    0     0     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0     0  1002 neg %cx
    0     0     0     0       0     1  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0    -1                           1002 neg %cx
    0     0     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0     1  1003 add $1, %cx
    0     0     0     0       0    -1  ------ Interrupt ------  ------ Interrupt ------
    0     0     0     0       0     0                           1003 add $1, %cx
    0     0     0     0       0     1  ------ Interrupt ------  ------ Interrupt ------
    1     0     0     0       0     1  1004 mov $1, 0(%fx,%bx,4)
    1     0     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1004 mov $1, 0(%fx,%bx,4)
    1     1     0     0       0     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     1     0       0     1  1005 mov %cx, turn
    1     1     1     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1005 mov %cx, turn
    1     1     0     0       0     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     1  1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     0                           1006 mov 0(%fx,%cx,4), %ax
    1     1     0     0       1     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     1  1007 test $1, %ax
    1     1     0     0       1     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     0                           1007 test $1, %ax
    1     1     0     0       1     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     1  1008 jne .fini
    1     1     0     0       1     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       1     0                           1008 jne .fini
    1     1     0     0       1     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     1  1009 mov turn, %ax
    1     1     0     0       1     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1009 mov turn, %ax
    1     1     0     0       0     1  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     1  1010 test %cx, %ax
    1     1     0     0       0     0  ------ Interrupt ------  ------ Interrupt ------
    1     1     0     0       0     0                           1010 test %cx, %ax
```

Case 4a: Thread 0 writes `turn` last, so Thread 1 enters first

python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 00000111111000000000000000001111111111111111111

Case 4b: Thread 1 writes `turn` last, so Thread 0 enters first

python x86.py -p peterson.s -M 100,104,turn,count -R ax,cx -c -a bx=0,bx=1 -P 00000011111100000000000000001111111111111111111

11. Now study the code for the ticket lock in ticket.s. Does it match the code in the chapter? Then run with the following flags: -a bx=1000,bx=1000 (causing each thread to loop through the critical section 1000 times). Watch what happens; do the threads spend much time spin-waiting for the lock?

1. Does it match the chapter? Yes.

Chapter (C)	ticket.s
myturn = FetchAndAdd(&lock->ticket);	mov $1, %ax / fetchadd %ax, ticket: ax = my ticket, and ticket goes up by 1
while (lock->turn != myturn) ;	mov turn, %cx / test %cx, %ax / jne .tryagain: spin until turn == my ticket
lock->turn = lock->turn + 1;	mov $1, %ax / fetchadd %ax, turn: turn goes up by 1

The only small difference is that the release uses fetchadd instead of a plain add. The result is the same, and it doesn't need to be atomic because only the lock holder updates turn.

2. Run:

python x86.py -p ticket.s -M ticket,turn,count -R ax,cx -c -a bx=1000,bx=1000

	Value
Final count	2000 ✓ correct
Instructions executed	99,563
Needed without spinning (13 per loop × 1000 × 2 threads, + 2 halt)	26,002
Spent spin-waiting	~73,500 (~74%)

3. Yes, the threads spend most of their time spinning. Trace excerpt:

ticket turn count Thread 0 Thread 1

12 11 12 1007 mov %ax, count

12 11 12 1008 mov $1, %ax ← T0 holds lock, turn not yet bumped

------ Interrupt ------

13	11	12	1001 fetchadd → my ticket = 12
13	11	12	1002–1004 turn=11 ≠ 12 → spin
13	11	12	1002–1004 spin … (rest of time slice)

Why:

- With the default interrupt interval (50), a thread is often interrupted while holding the lock. The other thread then wastes its whole time slice spinning on mov turn / test / jne.
- The ticket lock is FIFO, so the threads must take turns. After a thread releases, it immediately takes a new ticket, but if the other thread has an earlier ticket, the running thread has to spin until it's descheduled.

Conclusion: the ticket lock is correct (count = 2000) and fair (threads get the lock in order, so neither starves). But on a single CPU it's very inefficient: about 74% of the CPU time goes to spin-waiting, because waiting threads spin instead of giving up the CPU.

12. How does the code behave as you add more threads?

Command (each thread loops 1000 times):

python x86.py -p ticket.s -t <N> -M count -c -a bx=1000

Replace <N> with the number of threads, e.g. -t 4. -a bx=1000 applies to all threads.

Results:

Threads	count	instructions	needed (no spin)	wasted spinning	% wasted
1	1000	13,002	13,001	1	0%
2	2000	99,563	26,002	73,561	74%
3	3000	149,524	39,003	110,521	74%
4	4000	199,386	52,004	147,382	74%
6	6000	299,110	78,006	221,104	74%

Threads	count	instructions	needed (no spin)	wasted spinning	% wasted
8	8000	398,936	104,008	294,928	74%

Observations:

1. **Still correct.** count always equals threads × 1000, so mutual exclusion holds for any number of threads.
2. **Still fair.** The ticket lock serves threads in FIFO order, so no thread starves, even with 8 threads.
3. **Very inefficient once there are 2 or more threads.** About **74% of all instructions are spin-waiting**, and the total wasted work grows **linearly** with the number of threads (73K → 295K instructions).
4. **About one critical section per time slice.** Total instructions ≈ 50 × (number of increments), which is one 50-instruction time slice per increment. In each slice, the running thread finishes roughly one pass of the critical section. It then takes a new ticket and has to wait for **every other thread** to get its turn. Those threads aren't running, so it spins for the rest of its slice.
5. **Each thread waits longer as threads are added.** A thread's ticket comes after N − 1 others, so it has to wait through about N − 1 other time slices before it can enter again.

Conclusion: adding threads doesn't break the ticket lock, and it stays correct and fair. But on a single CPU, most of each time slice is wasted spinning, and the total waste grows with the number of threads. A better approach is to **yield** the CPU (yield.s) or sleep while waiting instead of spinning.

13. Now examine yield.s, in which a yield instruction enables one thread to yield control of the CPU (realistically, this would be an OS primitive, but for the simplicity, we assume an instruction does the task). Find a scenario where test-and-set.s wastes cycles spinning, but yield.s does not. How many instructions are saved? In what scenarios do these savings arise?

How yield.s differs: the acquire is the same xchg test-and-set, but when the lock is held, the thread runs yield and gives up the CPU immediately instead of looping back:

asm

```
xchg %ax, mutex
test $0, %ax
je  .acquire_done   # got the lock
yield               # lock held → give CPU to another thread
j   .acquire        # when rescheduled, try again
```

Scenario: 2 threads, bx=10, -i 50.

```
python x86.py -p test-and-set.s -M mutex,count -R ax,bx -c -a bx=10,bx=10 -i 50
python x86.py -p yield.s       -M mutex,count -R ax,bx -c -a bx=10,bx=10 -i 50
```

Thread 0 gets interrupted while holding the lock, in the middle of the critical section. Then Thread 1 runs:

	What Thread 1 does	Instructions used
test-and-set.s	spins on mov / xchg / test / jne for its entire time slice	50
yield.s	one failed xchg, then yield → CPU goes straight back to Thread 0	5

About 45 instructions are saved each time this happens. Thread 0 then finishes its critical section and releases the lock right away.

Total savings (total instructions, both programs give the correct count):

-i	bx=10: TAS	yield	saved	bx=100: TAS	yield	saved
5	298	306	−8	2998	3006	−8
11	222	222	0	2202	2202	0
15	358	276	82	3538	2796	742

-i	bx=10: TAS	yield	saved	bx=100: TAS	yield	saved
20	342	240	102	3010	2526	484
50	326	234	92	3354	2334	1020
100	222	222	0	3594	2274	1320

When the savings happen:

- When the lock holder is interrupted inside the critical section (it's preempted while holding the lock). With test-and-set, the waiting thread burns its whole time slice. With yield, it hands the CPU back after about 5 instructions.
- The longer the time slice, the bigger the savings, because there's more spinning to avoid. That's why -i 50 and -i 100 with many loops save the most (over 1,000 instructions, about 30–37%).
- No savings when there's no contention, for example with -i 11 or a slice long enough for each thread to finish. Neither program spins, so they're the same.
- Slightly worse with very short slices (-i ≤ 5). The spinner gets only a few instructions anyway, and the extra yield / j instructions cost a little more.

Conclusion: yielding avoids wasting a whole time slice when the lock holder has been preempted, so it's much more efficient than pure spinning. It still has costs: each failed attempt pays for a context switch, and with many threads, a waiting thread may be scheduled again and again just to yield. That's why real systems use queues and sleeping (as in the next part of the chapter).

14. Finally, examine test-and-test-and-set.s. What does this lock do? What kind of savings does it introduce as compared to test-and-set.s?

What the lock does: it adds a **plain read ("test")** before the atomic **test-and-set (xchg)**:

asm

```
.acquire
mov  mutex, %ax    # 1. TEST: just read the lock (normal load)
test $0, %ax
```

```
jne  .acquire      #    lock held → keep spinning on the READ only
mov  $1, %ax       # 2. lock looks free → now try the atomic TEST-AND-SET
xchg %ax, mutex
test $0, %ax
jne  .acquire      #    someone beat us to it → go back to reading
```

- While the lock is held, a waiting thread spins on a **normal read**, not on xchg.
- It tries the atomic xchg only when the lock **looks free**. The xchg is still needed, because another thread could grab the lock between the read and the swap.
- The critical section and the release (mov $0, mutex) are the same as in test-and-set.s.

What's saved:

- Far fewer atomic xchg operations. TTAS needs about 200–230, close to the minimum of one per acquire (200), while TAS uses 400–550. Nearly all of TAS's extra xchg calls happen while the lock is held and accomplish nothing.
- Why that matters on real hardware: xchg is an atomic write. On a multiprocessor, every write forces the cache line holding mutex to bounce between CPUs and invalidates the copies in other caches. A plain read can spin in the CPU's local cache without generating any bus traffic. So TTAS greatly reduces cache-coherence traffic, which makes spinning cheaper for the lock holder and for everyone else.

What isn't saved, in this simulator:

- The total instruction count goes up, not down. Each acquire takes 3 extra instructions (the initial read, test and jump), so even without contention one loop pass is 14 instructions instead of 11.
- TTAS still spin-waits, so it doesn't fix the wasted CPU time from Q6. Only yielding or sleeping does that (Q13).
- The simulator has one CPU and no caches, so it can't show the main benefit, which is less memory and cache traffic.

Conclusion: test-and-test-and-set is still a spin lock and is still correct. Its advantage is that it performs the expensive atomic operation only when the lock is likely free. That greatly reduces atomic writes (about 2× fewer here) and the cache traffic they cause on a real multicore machine, at the cost of a few extra instructions per acquire.

Chapter 29

Homework (Code) In this homework, you'll gain some experience with writing concurrent code and measuring its performance. Learning to build code that performs well is a critical skill and thus gaining a little experience here with it is quite worthwhile.

Questions 1. (timer.c) We'll start by redoing the measurements within this chapter. Use the call gettimeofday() to measure time within your program. How accurate is this timer? What is the smallest interval it can measure? Gain confidence in its workings, as we will need it in all subsequent questions. You can also look into other timers, such as the cycle counter available on x86 via the rdtsc instruction.

The smallest interval `gettimeofday()` can measure is 1 microsecond. That limit comes from its format: `struct timeval` only stores whole µs. The hardware can actually tick faster than that. I ran `timer.c` on a Linux x86-64 machine, and your numbers will vary a bit.

What the test does

1. Smallest step: calls `gettimeofday()` over and over until the value changes, then records the smallest jump it saw.
2. Back-to-back calls: checks how often two calls in a row return the exact same time.
3. Cost per call: times 1 million calls and averages them.
4. Accuracy check: times `usleep()` calls of known lengths.
5. Comparison: does the same with `clock_gettime(CLOCK_MONOTONIC)` and `rdtsc`.

Results

Timer	Smallest step	Cost per call
`gettimeofday`	1 µs	~26 ns
`clock_gettime(MONOTONIC)`	~21 ns (reports 1 ns)	~20 ns
`rdtsc` (2.1 GHz counter)	~26 cycles ≈ 12 ns	~12 ns

- Each call takes about 26 ns, which is much less than 1 µs. So 98.7% of back-to-back calls returned the same time. Anything shorter than about 1 µs shows up as 0 or 1.
- The sleep test shows the timer itself is accurate. The extra time comes from the OS waking the thread late:
  - `usleep(1)` measured as 76 µs
  - `usleep(1000)` measured as 1083 µs (+8%)
  - `usleep(100000)` measured as 100111 µs (+0.1%)

2. (Counter.c) Now, build a simple concurrent counter and measure how long it takes to increment the counter many times as the number of threads increases. How many CPUs are available on the system you are using? Does this number impact your measurements at all?

I'll build a lock-protected counter like the one in the chapter, then time it with 1 to 8 threads. I'll also rerun it pinned to a single CPU to see whether the CPU count matters.

The machine I used has 2 CPUs (2 cores, no hyperthreading, Xeon at 2.1 GHz). That number matters a lot: with a single lock, running on 2 CPUs was 3–4× slower than running on 1.

What `counter.c` does

1. The counter: one `int` protected by one `pthread_mutex_t`, as in the chapter (Fig. 29.2).
2. The workload: each thread increments it 1,000,000 times, so total work grows with the thread count.
3. Timing: `gettimeofday()` around thread creation and joining, which is over 10 ms, as Q1 suggested.
4. Correctness check: at the end it compares the final count with `threads × 1M`. Every run passed.
5. The comparison: I ran it normally, then again with `taskset -c 0` to force it onto one CPU.

Results (seconds)

Threads	Both CPUs	Pinned to 1 CPU
1	0.019	0.021
2	0.161	0.044
4	0.300	0.078
8	0.536	0.146

What this shows

- One thread: about 19 ns per increment, the cost of a lock and unlock with no one else competing.
- Two or more threads on 2 CPUs: about 75 ns per increment, roughly 4× slower per operation. Ideally 2 threads would finish 2M increments in the same 0.019 s as one thread does 1M. They took 0.16 s instead. This is the chapter's point that a single-lock counter doesn't scale.
- Why it gets slower: the lock's cache line keeps moving between the two cores. Threads also fight over the lock and sometimes have to sleep in the kernel and be woken up again.
- Pinned to 1 CPU: the cost stays near the single-thread level (about 18–22 ns per increment), even with 8 threads. Only one thread runs at a time, so the cache line never

moves. A thread usually gets through a lot of increments before the scheduler switches it out, so it rarely finds the lock held.
- More threads than CPUs: beyond 2 threads, total time just grows in proportion to the work. With only 2 cores, the extra threads take turns and never run in parallel.

The CPU count affects your measurements in two ways. It sets how many threads can actually run (and compete) at once, and here using more CPUs made things slower. For the later questions (the approximate counter and others), include `nproc` in your results. The interesting effects only show up once threads ≥ 2 and there are at least 2 CPUs. If your own machine has more cores, run this up to that many threads to see the full curve. I didn't repeat each run, so expect timings to move by about ±10%.

3. (approx.c)Next, build a version of the approximate counter. Once again, measure its performance as the number of threads varies, as well as the threshold. Do the numbers match what you see in the chapter?

I'll build the approximate counter from the chapter (Fig. 29.4: one local counter per CPU, each with its own lock, plus a global counter), then measure it two ways: varying the thread count with the threshold fixed, and varying the threshold with the thread count fixed.

Yes, the pattern matches the chapter: the approximate counter scales well when the threshold is large and does badly when it's small. The absolute numbers don't match, because my 2-CPU machine is very different from the chapter's 4-CPU one.

What `approx.c` does

1. Structure: like the chapter's Fig. 29.4, with one local counter and lock per CPU (2 here) and one global counter and lock. Thread `i` uses local counter `i % ncpus`.
2. Update: lock the local counter and add to it. Once it reaches S, move the total into the global counter under the global lock, then reset the local one.
3. One change from the book: each local counter is padded to its own 64-byte cache line. In the book's array layout, both CPUs' counters sit on the same cache line, so the cores would still fight over it ("false sharing").
4. Workload: same as Q2, 1M increments per thread, on 2 CPUs.

Experiment A: vary threads, S = 1024 (seconds)

Threads	Precise (Q2)	Approximate
1	0.019	0.019
2	0.161	0.021
4	0.300	0.042
8	0.536	0.188

- With 2 threads on 2 CPUs, the job takes 0.021 s, almost the same as 1 thread. That means twice the work in the same time, close to perfect scaling and about 8× faster than the precise counter.
- With 3, 5 or 7 threads, results are worse and noisier (for example 0.079 s with 3 threads). Two threads end up sharing one local lock, so contention comes back. Keeping threads ≤ CPUs, or using an even split, avoids this.

Experiment B: vary S, 4 threads (seconds)

S	1	2	4	8	32	1024
Time	0.410	0.165	0.093	0.054	0.042	0.049

- S = 1: the global lock is taken on every update, so it's even slower than the precise counter (0.41 s vs 0.30 s), because it pays for two locks instead of one.
- S around 32 or more: the time levels off. The global lock is now rarely touched, so the cores mostly work independently.
- Accuracy trade-off: `get()` only reads the global count, so it can fall short by up to `ncpus × (S − 1)`. With S = 1024 and 1 thread it returned 999,424 instead of 1,000,000. Larger S means faster updates but a less accurate reading.

Compared with the chapter

- Same overall pattern: the precise counter falls apart as threads are added, the approximate counter with large S stays nearly flat, and the "time vs. S" curve drops steeply and then levels off, as in Fig. 29.6.
- Smaller slowdown here: as far as I recall, the book reports about 0.03 s for one thread and over 5 s for two on the precise counter. Here the slowdown is only about 8× (0.019 s to 0.161 s). Today's Linux mutexes are much cheaper under contention than the ones the book used, and I only have 2 cores to compete.
- Less room for a big win: with 2 CPUs, there's simply less parallelism to gain. The book's 4-CPU machine shows a bigger gap between the two counters.

4. (hoh.c) Build a version of a linked list that uses hand-over-hand locking [MS04], as cited in the chapter. You should read the paper first to understand how it works, and then implement it. Measure its performance. When does a hand-over-hand list work better than a standard list as shown in the chapter?

How it works: each node has its own lock. While walking the list, you lock the next node before unlocking the current one, so several threads can move through the list at once, one behind another.

Results (1000-node list, 2 CPUs; hand-over-hand time ÷ standard time, so below 1× means hand-over-hand is faster):

Workload	1 thread	2 threads	4 threads
Cheap nodes	10.5×	2.3×	7.8×
Expensive nodes	0.88×	0.49×	0.61×

When hand-over-hand is better: only when visiting each node is expensive (for example, slow key comparisons) and you have about as many threads as CPUs. Otherwise, locking and unlocking at every node costs more than the parallelism gains, so the single-lock list wins. With more threads than CPUs, a thread that's switched out while holding a node lock blocks every thread behind it.

5. (bst.c) Pick your favorite data structure, such as a B-tree or other slightly more interesting structure. Implement it, and start with a simple locking strategy such as a single lock. Measure its performance as the number of concurrent threads increases.

I'll do a binary search tree with one global lock. I'll prefill it with random keys, then time a mix of lookups and inserts at 1 to 8 threads.

I picked a binary search tree with one global lock. It doesn't scale at all: 1 thread is fastest, and adding threads cuts throughput by half or more.

Setup: the tree starts with 100k random keys. Random insert order keeps it roughly balanced, about 20–25 levels deep. Each thread does 500k operations with random keys, on 2 CPUs.

Throughput (million ops/sec, higher is better)

Threads	10% inserts	50% inserts
1	5.20	2.96
2	1.33	0.59
4	2.44	1.19
8	2.44	1.11

What this shows:

- No scaling. One lock means only one thread is ever in the tree, so extra threads add overhead and no parallelism.

- 2 threads is the worst case. Both run at the same time on the 2 CPUs and keep fighting over the lock. The lock's cache line and the top of the tree keep moving between the cores, and losing threads go to sleep in the kernel and wake up again. Throughput drops about 4–5×.
- 4 and 8 threads do a bit better than 2 (but still worse than 1). With more threads than CPUs, the scheduler often runs a thread for a whole time slice while the others wait. It's closer to taking turns than to constant fighting.
- Inserts make it worse. They write to the tree, so the cache lines involved move between cores even more.

5. (bst2.c) Finally, think of a more interesting locking strategy for this favorite data structure of yours. Implement it, and measure its performance. How does it compare to the straightforward locking approach?

I'll compare the Q5 single-lock tree with two better strategies: a reader-writer lock, so lookups can run in parallel, and hand-over-hand locking down the tree, so operations in different subtrees don't block each other.

I tried two smarter locking strategies on the BST. Each one beats the single lock only for certain workloads, and with 1 thread the single lock is always fastest.

The strategies:

1. Reader-writer lock: lookups can share the tree, while inserts need it to themselves.
2. Hand-over-hand down the tree: each node has its own lock. You lock the child before releasing the parent, so two operations only block each other while their paths overlap.
   Throughput with 2 threads on 2 CPUs (million ops/sec; higher is better)

Workload	Single mutex	Reader-writer lock	Hand-over-hand
0% inserts	1.04	6.08	1.16
10% inserts	0.74	0.52	0.98
50% inserts	0.39	0.22	1.03
1 thread, 10% inserts	3.73	3.60	1.85

What this shows:

- The reader-writer lock is the clear winner with lookups only. It's 6× faster than the mutex at 2 threads, and it keeps improving with more threads (7.55 at 8 threads). It's the only version here that actually scales.
- But a few inserts ruin it. With even 10% inserts it's slower than the plain mutex. Each insert has to wait for all readers to leave, and every lookup still writes to the lock's shared counter, so that cache line keeps moving between the cores anyway.
- Hand-over-hand is best once there are inserts. At 50% inserts it's about 2.6× faster than the mutex, because inserts into different subtrees run in parallel.
- But it costs a lot with one thread. Taking about 20 locks per operation makes it about half as fast as the single lock. Every operation also passes through the root's lock, so the top of the tree is still a bottleneck.
- More threads than CPUs hurts both. At 4 and 8 threads, a thread can be switched out while holding a lock, and the others pile up behind it.

  Compared with the straightforward approach: use a reader-writer lock if the workload is almost all reads. Use hand-over-hand if there's a meaningful share of writes and you have about as many threads as CPUs. Otherwise the single lock is simpler and often just as fast. As the chapter puts it, more concurrency isn't necessarily faster.

  As before, these are single runs on 2 CPUs. A machine with more cores should show bigger gains for both strategies.